

#Downloading and Running an LLM

```
!pip install \
transformers==4.46.3 \
huggingface_hub==0.26.2 \
accelerate==1.1.1 \
tokenizers==0.20.3 \
safetensors
```

```
Requirement already satisfied: transformers==4.46.3 in /usr/local/lib/python3.12/dist-packages (4.46.3)
Requirement already satisfied: huggingface_hub==0.26.2 in /usr/local/lib/python3.12/dist-packages (0.26.2)
Requirement already satisfied: accelerate==1.1.1 in /usr/local/lib/python3.12/dist-packages (1.1.1)
Requirement already satisfied: tokenizers==0.20.3 in /usr/local/lib/python3.12/dist-packages (0.20.3)
Requirement already satisfied: safetensors in /usr/local/lib/python3.12/dist-packages (0.8.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (3.29.4)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (26.2)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (2022.10.31)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (2.32.4)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (4.67.3)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub==0.26.2) (2024.10.1)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub==0.26.2) (4.12.2)
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from accelerate==1.1.1) (5.9.5)
Requirement already satisfied: torch>=1.10.0 in /usr/local/lib/python3.12/dist-packages (from accelerate==1.1.1) (2.11.0+cpu)
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->accelerate==1.1.1) (75.0.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->accelerate==1.1.1) (1.13.3)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->accelerate==1.1.1) (3.3)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.10.0->accelerate==1.1.1) (3.1.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->transformers==4.46.3) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->transformers==4.46.3) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->transformers==4.46.3) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->transformers==4.46.3) (2024.12.14)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.10.0->accelerate==1.1.1) (3.1.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch>=1.10.0->accelerate==1.1.1) (3.0.2)
```

```
# It deletes the partially downloaded phi 3 files and starts downloading from strating
!rm -rf ~/.cache/huggingface/hub/models--microsoft--Phi-3-mini-4k-instruct
```

```
# Importing model and tokenizer
from transformers import AutoModelForCausalLM, AutoTokenizer

model_name="microsoft/Phi-3-mini-4k-instruct"
tokenizer=AutoTokenizer.from_pretrained(model_name)

model=AutoModelForCausalLM.from_pretrained(model_name,
                                             device_map="auto",
                                             torch_dtype="auto",
                                             trust_remote_code=True
                                             )
```

Loading checkpoint shards: 100%

2/2 [00:01<00:00, 1.69it/s]

```
# prompt
# <|assistant|> this means this the end of prompt now the model has to generate

prompt = "Write an email apologizing to Sarah for the tragic gardening mishap. Explain how it happened.<|assistant|>"

# tokenizer converts the text into token ids , tokenizer it retruns the output as a dictionary containing of token ids and
# we are taking only the input ids and moving to cpu becuase the model also running on cpu

input_ids = tokenizer(prompt,
return_tensors="pt").input_ids.to("cpu")

# Instead of predicting just one next token, generate() repeatedly predicts the next token until it reaches the specified l
# input_ids=input_ids ==> Start generating using these input tokens
# max_new_tokens=20 ==> No.of.new tokens to generate

generation_output = model.generate(
input_ids=input_ids,
max_new_tokens=20
)

# Converting the newly generated token ids back to tokens (text)
# gen_txt[0] ==> generate retruns a batch of outputs , selecting the first generated output
print(tokenizer.decode(gen_txt[0]))
```

```
print(tokenizer.decode(generation_output[0]))
```

tokenizer.model: 100%

500k/500k [00:00<00:00, 33.7MB/s]

Loading checkpoint shards: 100%

2/2 [00:00<00:00, 2.38it/s]

Write an email apologizing to Sarah for the tragic gardening mishap. Explain how it happened.<|assistant|> Subject: Heartfe

Dear

```
# It has Pytorch tensor values
print(input_ids)
```

```
tensor([[14350, 385, 4876, 27746, 5281, 304, 19235, 363, 278, 25305,
         293, 16423, 292, 286, 728, 481, 29889, 12027, 7420, 920,
         372, 9559, 29889, 32001]])
```

#This prints (each token is on a separate line):

```
for id in input_ids[0]:
    print(tokenizer.decode(id))
```

```
Write
an
email
apolog
izing
to
Sarah
for
the
trag
ic
garden
ing
m
ish
ap
.
Exp
lain
how
it
happened
.
<|assistant|>
```

#This shows the input tokens as well as the output tokens

```
print(generation_output)
```

```
tensor([[14350, 385, 4876, 27746, 5281, 304, 19235, 363, 278, 25305,
         293, 16423, 292, 286, 728, 481, 29889, 12027, 7420, 920,
         372, 9559, 29889, 32001, 3323, 622, 29901, 17778, 29888, 2152,
         6225, 11763, 363, 278, 19906, 292, 341, 728, 481, 13,
         13, 13, 29928, 799]])
```

Decodes the token ids back to text

```
print(tokenizer.decode(3323))
print(tokenizer.decode(622))
print(tokenizer.decode([3323, 622]))
print(tokenizer.decode(29901))
```

```
Sub
ject
Subject
:
```

Comparing Trained LLM Tokenizers

BERT Base Uncased

```
# Import the tokenizer automatically suitable for Model
from transformers import AutoTokenizer
```

```
# This is simply a list of background colors.
#Each token will be displayed in a different color.
```

```

colors_list = [
    '102;194;165',
    '252;141;98',
    '141;160;203',
    '231;138;195',
    '166;216;84',
    '255;217;47'
]

# A function with two arguments
# sentence = input text , tokenizer name
def show_tokens(sentence, tokenizer_name):

    # downloads and loads whichever tokenizer you specify.
    tokenizer = AutoTokenizer.from_pretrained(tokenizer_name)

    # tokenizer splits the text into tokens
    # tokens into token ids
    # stores only input ids
    token_ids = tokenizer(sentence).input_ids

    # enumerate gives the token ids with their position
    # idx = the index position of the token
    # t = token ids
    for idx, t in enumerate(token_ids):
        print(

            # \x1b = Ascii escape character it tells to start formatting text
            # 0;30;48;2 = This controls the style. ( 0=Reset formatting style after a token was generated with color , 30
            # Pick a background color based on the current token position.
            # m = Apply the formatting
            f'\x1b[0;30;48;2;{colors_list[idx % len(colors_list)]}m'

            # Converts token ids back to text
            + tokenizer.decode(t)

            # End of formatting
            + '\x1b[0m',
            end=' '

        )

    # Input text
    text = """
English and CAPITALIZATION
show_tokens False None elif == >= else: two tabs:" " Three tabs:
" "
12.0*50=600

"""

# Function call with input and tokenizer name
show_tokens(text, "bert-base-uncased")

```

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 1.85kB/s]

config.json: 100% 570/570 [00:00<00:00, 47.4kB/s]

vocab.txt: 232k/? [00:00<00:00, 2.77MB/s]

tokenizer.json: 166k/? [00:00<00:00, 5.08MB/s]

[CLS] english and capital ##ization show _ token ##s false none elif == >= else : two tabs:" " Three tabs: " "

BERT Base Cased

show_tokens(text, "bert-base-cased")

tokenizer_config.json: 100% 49.0/49.0 [00:00<00:00, 2.21kB/s]

config.json: 100% 570/570 [00:00<00:00, 22.8kB/s]

vocab.txt: 213k/? [00:00<00:00, 12.3MB/s]

tokenizer.json: 436k/? [00:00<00:00, 12.3MB/s]

[CLS] English and CA ##PI ##TA ##L ##I ##Z ##AT ##ION show _ token ##s F ##als ##e None el ##if == >= else : two tabs:" " Three tabs: " "

GPT-2

show_tokens(text, "gpt2")

```

tokenizer_config.json: 100%                26.0/26.0 [00:00<00:00, 2.43kB/s]
config.json: 100%                          665/665 [00:00<00:00, 53.3kB/s]
vocab.json:      1.04M/? [00:00<00:00, 18.7MB/s]
merges.txt:      456k/? [00:00<00:00, 15.6MB/s]
tokenizer.json:   1.36M/? [00:00<00:00, 29.1MB/s]

English and CAPITALIZATION
show_tokens False None elif == >= else : two tabs : " " Three tabs :
" "
12 . 0 * 50 = 600

```

FLAN-T5

```
show_tokens(text, "google/flan-t5-base")
```

```

tokenizer_config.json: 2.54k/? [00:00<00:00, 198kB/s]
spiece.model: 100%                792k/792k [00:00<00:00, 11.5MB/s]
tokenizer.json: 2.42M/? [00:00<00:00, 61.9MB/s]
special_tokens_map.json: 2.20k/? [00:00<00:00, 165kB/s]
English and CAPITALIZATION show_tokens False None elif == >= else : two tabs : " " Three tabs : " " 12 . 0

```

StarCoder

```
show_tokens(text, "bigcode/starcoder2-15b")
```

```

tokenizer_config.json: 7.88k/? [00:00<00:00, 553kB/s]
vocab.json: 777k/? [00:00<00:00, 23.5MB/s]
merges.txt: 442k/? [00:00<00:00, 18.4MB/s]
tokenizer.json: 2.06M/? [00:00<00:00, 56.0MB/s]
special_tokens_map.json: 100%                958/958 [00:00<00:00, 76.2kB/s]

English and CAPITALIZATION
show_tokens False None elif == >= else : two tabs : " " Three tabs :
" "
12 . 0 * 50 = 600

```

Galactica

```
show_tokens(text, "facebook/galactica-6.7b")
```

```

tokenizer_config.json: 100%                166/166 [00:00<00:00, 10.6kB/s]
tokenizer.json: 2.14M/? [00:00<00:00, 15.9MB/s]
special_tokens_map.json: 100%                3.00/3.00 [00:00<00:00, 129B/s]

English and CAPITALIZATION
show_tokens False None elif == >= else : two tabs : " " Three tabs :
" "
12 . 0 * 50 = 600

```

Phi-3 and Llama

```
show_tokens(text, "microsoft/Phi-3-mini-4k-instruct")
```

```

English and CAPITALIZATION
show_tokens False None elif == >= else : two tabs : " " Three tabs :
" "

```

```
1 2 . 0 * 5 0 = 6 0 0
```

Start coding or [generate](#) with AI.

TOKEN EMBEDDINGS

```
from transformers import AutoModel, AutoTokenizer

# Load a tokenizer
tokenizer = AutoTokenizer.from_pretrained("microsoft/deberta-base")

# Load a language model
model = AutoModel.from_pretrained("microsoft/deberta-v3-xsmall")

# Tokenize the sentence
tokens = tokenizer('Hello world', return_tensors='pt')

# Process the tokens
# input_ids → token numbers
# ** for attention_mask → tells the model which positions are real tokens
# sends token IDs and attention mask into the DeBERTa model
# gets the final contextual embedding vector for every token.
output = model(**tokens)[0]

# shows how many sentences, how many tokens, and how many numbers represent each token.
print(output.shape)
```

```
torch.Size([1, 4, 384])
```

```
# Raw output of a language model
print(output)
```

```
tensor([[[[-3.4816,  0.0861, -0.1819, ..., -0.0612, -0.3911,  0.3017],
          [ 0.1898,  0.3208, -0.2315, ...,  0.3714,  0.2478,  0.8048],
          [ 0.2071,  0.5036, -0.0485, ...,  1.2175, -0.2292,  0.8582],
          [-3.4278,  0.0645, -0.1427, ...,  0.0658, -0.4367,  0.3834]]],
        grad_fn=<NativeLayerNormBackward0>)
```

```
# The tokenizer split your input into tokens, including any special tokens
# tokens['input_ids'][0]: it is the no.of input sentences in this case we have only one so 0
for token in tokens['input_ids'][0]:
```

```
    #decoding the token ids
    print(tokenizer.decode(token))
```

```
[CLS]
Hello
world
[SEP]
```

TEXT EMBEDDINGS

```
# A python library used for sentence/text embeddings
# SentenceTransformer = Automatically loads a pretrained sentence embedding model.
from sentence_transformers import SentenceTransformer

# Load model
model = SentenceTransformer("sentence-transformers/all-mpnet-base-v2")

# Convert text to text embeddings
# encode () = Converts text into a numerical vector. One vector represents the entire sentence
vector = model.encode("Best movie ever!")

# shape tells us the dimensions (size) here 768 the values inside the vector
print(vector.shape)
print(vector)
```


Loading weights: 100%

199/199 [00:00<00:00, 4132.46it/s]

(768,)
[-2.02204362e-02 4.57697175e-02 -1.26636680e-02 -3.37981712e-03
-2.54916237e-03 -3.31098144e-03 -4.58366238e-02 2.32390836e-02
-3.12584974e-02 -3.15589011e-02 -2.19784174e-02 3.67827923e-03
3.39107984e-03 -1.88130755e-02 -2.65821852e-02 -1.45891728e-02
3.20067517e-02 -3.66492081e-03 1.75410118e-02 5.43209016e-02
-3.96661721e-02 9.93632246e-03 -3.27204466e-02 -1.62909776e-02
6.36536581e-03 -1.24458848e-02 1.04737468e-02 3.08674239e-02
7.47450348e-03 -3.86319943e-02 -6.19925074e-02 -3.25308926e-02
-1.40267909e-02 5.16587272e-02 1.69651696e-06 -1.74263827e-04
-1.27881835e-03 -4.15052250e-02 5.00354404e-03 4.58600707e-02
-2.85336580e-02 5.02370968e-02 -1.75184160e-02 -3.98945122e-04
1.63310990e-02 5.89937754e-02 -2.21730005e-02 -4.00975011e-02
-6.60441071e-03 -2.97633186e-02 -3.18221338e-02 -1.77945402e-02
-1.54363886e-02 -3.44514363e-02 6.25760406e-02 2.34382786e-03
3.44158150e-02 -8.25463887e-03 7.44930431e-02 -3.81284840e-02
-6.80808201e-02 -4.23290022e-02 -1.83172002e-02 -2.44083814e-02
1.99338831e-02 -2.06878372e-02 2.39973068e-02 3.49855199e-02
-6.95871115e-02 4.31547910e-02 1.57101154e-02 -4.87721944e-03
6.38265237e-02 2.81548444e-02 5.08511923e-02 -2.83364672e-02
-4.60657254e-02 6.47141039e-02 6.61476478e-02 -2.46198382e-02
-6.05491735e-02 -3.04247858e-03 6.28791302e-02 2.83486620e-02
5.71174733e-02 6.52742106e-03 -6.45677838e-03 1.11732865e-02
2.18259990e-02 -1.60584133e-02 3.98909673e-03 -2.04688543e-03
3.96740483e-03 4.18876335e-02 -2.90660802e-02 -4.19958755e-02
4.34483215e-02 4.26485017e-02 7.53306411e-03 -3.28948535e-02
-8.28775112e-03 2.01880764e-02 1.37379384e-02 1.38778230e-02
4.24899273e-02 2.69808136e-02 -1.83731169e-02 -3.71770374e-02
3.31590287e-02 2.42859740e-02 -3.89447547e-02 2.28922628e-03
5.35068847e-02 6.79783225e-02 -9.69997421e-03 -6.89620944e-03
8.24520290e-02 -4.85093566e-03 -1.19631952e-02 1.12272063e-02
-1.84042472e-02 -9.23715997e-03 -2.06847917e-02 2.53276899e-02
-1.11306412e-02 -4.08678642e-03 -2.30704062e-02 -2.89187431e-02
4.45918739e-02 -3.02654109e-03 -4.13767956e-02 5.19979149e-02
4.95346216e-03 -6.28712177e-02 3.07493508e-02 4.33227532e-02
-2.99513489e-02 -2.41317484e-03 3.73285934e-02 -4.21253359e-03
3.35869826e-02 -4.57396023e-02 -7.57046491e-02 -1.43271209e-05
3.77049781e-02 1.17524546e-02 6.80495473e-03 -2.76922062e-03
2.55940538e-02 -4.55860943e-02 3.38432565e-02 -2.46407632e-02
-1.58958342e-02 -9.64218378e-03 -1.74003113e-02 1.95757300e-02
-1.72167197e-02 -4.00132760e-02 8.84149317e-03 5.63183334e-03
-5.64631745e-02 4.18703556e-02 6.14107288e-02 -2.33448055e-02
-6.02262393e-02 2.65606251e-02 -8.74001440e-03 -2.54317671e-02
-5.78649864e-02 -1.63435028e-03 -3.57386321e-02 8.78347363e-03
-1.86852198e-02 -2.27694120e-02 -1.64403692e-02 9.16374614e-04
-2.75901728e-03 -2.11251210e-02 1.17958635e-02 6.54988140e-02
-1.34429485e-02 -5.85039565e-03 -2.26886198e-02 -4.69191140e-03
-6.56704325e-03 -5.58437174e-03 -2.37321351e-02 2.63367174e-03
2.79200431e-02 1.75014585e-02 7.05148354e-02 2.04405584e-03
3.96885984e-02 -7.11204764e-03 -5.53258648e-03 -1.76759828e-02
-9.80821531e-03 4.24955739e-03 5.34048453e-02 7.13995174e-02
1.45237464e-02 6.63728500e-03 1.84460226e-02 -3.81756462e-02
-1.90555844e-02 -5.48271872e-02 -1.84072219e-02 -4.95260581e-02
-4.00507078e-02 8.68838560e-03 -2.48502102e-02 4.53157276e-02
2.03235634e-02 -3.21139842e-02 1.56295318e-02 3.62754315e-02
6.59926534e-02 3.26602571e-02 -8.78327936e-02 9.44340080e-02
2.16406398e-02 -8.00881628e-03 -5.13048517e-03 1.49668865e-02
-9.38003324e-03 -1.62822157e-02 -8.78272054e-04 -2.41658017e-02
7.28240758e-02 5.06999157e-02 -7.54104787e-03 2.89576352e-02
6.94771558e-02 -3.20685804e-02 8.36075470e-03 2.72287801e-02
4.86844368e-02 2.09293980e-02 -4.85208444e-03 -1.35733649e-01
5.91988340e-02 3.52462940e-02 -3.54929129e-03 5.53837558e-03
-7.42439693e-03 -1.85083635e-02 -2.47552097e-02 3.68198268e-02
4.10625041e-02 4.83450294e-03 -5.03980666e-02 -7.56626204e-03
-7.34616145e-02 -1.37477517e-02 -4.96552140e-02 -5.77377416e-02
2.30375770e-02 1.72315389e-02 -1.10097872e-02 -5.00842556e-03
3.73133607e-02 -1.22197289e-02 3.67946364e-02 3.23757641e-02
3.03839426e-02 3.12437974e-02 1.34678381e-02 2.09657801e-03
9.05989110e-03 3.20697837e-02 4.30424064e-02 1.61135048e-02
-1.97644881e-03 4.51486744e-02 -2.61760154e-03 -2.10353043e-02
1.23143448e-02 2.19331216e-02 -7.48108104e-02 -1.87375285e-02
5.77955879e-03 -1.66842695e-02 3.49258073e-02 1.69625487e-02
-1.29548423e-02 2.42918506e-02 1.33142872e-02 -6.13085693e-03
1.24596031e-02 -6.53127357e-02 -1.60475224e-02 -4.14024852e-02
3.02160401e-02 3.15758958e-02 2.17898525e-02 9.32295527e-03
-6.24403991e-02 -2.23614853e-02 -5.48127666e-02 -7.60868862e-02
-4.92596477e-02 3.03733274e-02 -1.11714425e-03 -2.49408223e-02
4.89795133e-02 -2.57985871e-02 -4.59413007e-02 -8.69534165e-03
-1.24728261e-02 2.25807875e-02 -2.81891096e-02 5.30083813e-02
-8.50981474e-02 -1.57342553e-02 -2.83088349e-02 1.50961112e-02
-3.79605568e-03 -6.45768493e-02 2.63364222e-02 3.87118496e-02
-4.34798218e-04 -1.71438679e-02 -3.64411175e-02 2.42591631e-02
-2.05201600e-02 1.98236108e-02 3.23042348e-02 -5.71224978e-03
4.89671156e-02 1.66042447e-02 -3.73704033e-03 -3.16586420e-02
5.87326065e-02 -3.70643660e-02 -9.46532376e-03 1.18760923e-02
-5.57537973e-02 -2.59903651e-02 -7.00724348e-02 4.13720198e-02
9.44962353e-03 8.65645707e-03 -2.11737175e-02 -3.13928537e-02
-4.39829975e-02 -2.41709929e-02 -6.17116913e-02 -7.56191686e-02

```
-2.43743733e-02 5.23748770e-02 9.59638413e-03 2.09737252e-02
-5.92921861e-03 2.82211369e-03 2.52180528e-02 -5.18828677e-03
-2.51272582e-02 -2.08314806e-02 -1.32330926e-02 -2.14497577e-02
4.04746970e-03 2.10042316e-02 2.12112151e-04 4.75079566e-02
6.97088093e-02 -1.95958167e-02 -5.54395616e-02 3.15217376e-02
1.19434865e-02 -3.56129557e-02 -4.14410653e-03 -1.77257985e-03
-6.62557557e-02 3.22467051e-02 -2.87152771e-02 -2.36792490e-02
-1.68482382e-02 -1.88507587e-02 -1.20192422e-02 7.31731488e-05
3.12583372e-02 4.72688349e-04 -6.26346422e-03 -3.22463624e-02
-2.92951316e-02 2.90422924e-02 -1.47413611e-02 2.43107546e-02
-1.26741473e-02 7.43111521e-02 6.77752867e-02 -6.38448028e-03
-8.33839830e-03 5.84009895e-03 3.46759818e-02 -7.89607875e-03
-9.79562476e-03 -3.47787403e-02 -5.03329560e-02 -6.80285646e-03
-2.32680119e-03 1.63682550e-01 3.31019871e-02 7.22549483e-02
6.55808076e-02 -4.43228666e-04 7.60614052e-02 3.29805538e-02
-2.50527300e-02 4.97035123e-03 -7.32460194e-06 1.77375798e-03
-4.73156944e-02 1.24154650e-02 -1.79417357e-02 -3.96824535e-03
-1.64654423e-02 -4.98489067e-02 -4.93970960e-02 -1.77263319e-02
-5.79471420e-03 5.12048975e-02 6.15340024e-02 -2.02966183e-02
4.10276502e-02 7.97440298e-03 1.60991319e-03 2.73910072e-03
2.56858617e-02 3.49120945e-02 3.48256081e-02 2.99327280e-02
-2.05291882e-02 -3.16913277e-02 -4.00760910e-03 -6.82936460e-02
2.53672320e-02 1.60931461e-02 3.90184904e-03 -1.00663421e-03
-4.16301638e-02 2.39276513e-02 1.93548966e-02 8.67203251e-03
-4.15939726e-02 -2.54745241e-02 -2.57344265e-02 -2.41258219e-02
1.68338802e-03 -1.94079510e-03 -1.33332354e-03 -2.77865864e-02
9.17956699e-03 -5.63237183e-02 6.82893246e-02 2.38570943e-02
3.62704992e-02 -9.64246690e-03 2.39587370e-02 2.71281619e-02
9.78971571e-02 3.71735394e-02 1.77358594e-02 -4.09085080e-02
5.76262511e-02 4.53530215e-02 -7.39420997e-03 -1.50913354e-02
1.35780703e-02 -3.29059921e-02 -2.47588288e-02 -1.26150856e-03
6.75928174e-03 -1.91759877e-02 -6.33700704e-03 -7.85873234e-02
2.50525288e-02 -9.13120657e-02 3.05409450e-02 -2.07534079e-02
-6.91623567e-03 -1.69420359e-03 1.80585179e-02 -9.01441462e-03
6.07969649e-02 -7.59931654e-02 -3.42036337e-02 2.36437228e-02
-6.94926409e-03 1.95292495e-02 -6.66117296e-03 -4.78437310e-03
-1.23866824e-02 -2.50967331e-02 -6.64266050e-02 -3.37183587e-02
2.51411181e-02 -4.08467278e-02 6.33135587e-02 -1.46272192e-02
1.37926899e-02 8.56993906e-03 -2.74824612e-02 8.24874416e-02
-3.33503969e-02 -1.88081674e-02 1.64880771e-02 2.23316830e-02
-2.33952515e-02 -1.54916309e-02 1.58764720e-02 1.32024847e-02
-3.13520618e-02 -3.49489935e-02 4.14551683e-02 2.04285011e-02
-4.56377082e-02 -6.66561127e-02 -1.16886310e-02 2.10026726e-02
9.88326501e-03 -6.19194796e-03 -4.83619906e-02 2.68508922e-02
1.07401162e-02 -1.21481670e-03 3.40227485e-02 -4.31549852e-04
7.58249760e-02 1.72218215e-02 -1.55711947e-02 1.66777417e-03
9.24158171e-02 1.51794814e-02 -4.27687652e-02 1.10518120e-01
2.41132136e-02 2.22330843e-03 1.09921750e-02 1.45183317e-02
-5.40816709e-02 -2.92326603e-03 -1.81705710e-02 -6.50137896e-03
8.84761959e-02 -4.08635363e-02 -2.46939640e-02 -5.75014716e-03
-3.22965011e-02 -2.96046883e-02 1.48974459e-05 6.85957726e-03
1.28586907e-02 -8.72264337e-03 -1.48365367e-02 -7.02032410e-33
-4.22565406e-03 -7.70923123e-02 -6.34941971e-03 1.88374091e-02
-1.08135357e-01 6.80670235e-03 1.62729416e-02 1.74701195e-02
-3.21585201e-02 -1.56154856e-03 -1.21566048e-02 1.36199770e-02
2.77593192e-02 -9.20694415e-03 -4.14167382e-02 -2.56593302e-02
-1.12561211e-02 4.94263545e-02 -2.78245821e-03 9.62397666e-04
4.10977900e-02 2.60311365e-02 4.17774245e-02 1.70260575e-02
-3.55315730e-02 3.93776670e-02 -8.13048333e-02 2.14785780e-03
6.34984374e-02 3.19657661e-02 5.42129464e-02 2.19714915e-04
-1.19869988e-02 -4.00769487e-02 6.91371784e-02 -2.37347931e-02
1.84541252e-02 5.91237098e-02 -1.81721263e-02 -3.54841277e-02
-7.07990676e-02 -3.47006917e-02 2.24270415e-03 -6.55508637e-02
-3.94934788e-02 1.90424975e-02 -1.63911916e-02 4.02782485e-02
3.15000303e-03 1.21331513e-02 -6.33518621e-02 -6.07369980e-03
-1.06719639e-02 -9.25485045e-02 -1.00543648e-02 -3.12487762e-02
-1.62010286e-02 1.15962503e-02 -7.22086877e-02 3.07976790e-02
-4.02981080e-02 -4.38704714e-02 7.42032938e-03 -1.51421288e-02
-9.55188926e-03 3.73820812e-02 5.66349551e-02 -8.54418147e-03
4.58036491e-04 -3.60886082e-02 -6.04566857e-02 -4.95447079e-03
-1.58476327e-02 -3.05450405e-03 4.35594656e-02 -3.49932499e-02
-8.74717534e-02 -2.11461075e-02 4.69009802e-02 1.14758767e-03
-9.70447995e-03 1.26246409e-02 3.87018062e-02 1.04797976e-02
3.05910371e-02 -4.95131500e-03 -2.14421377e-02 1.85080320e-02
-4.06456701e-02 9.62931139e-04 -7.90950283e-02 -1.98710598e-02
-6.67666737e-03 -1.66491754e-02 2.2318750e-02 -6.31384403e-02
-2.61657760e-02 -5.00156768e-02 -1.82532314e-02 2.92074904e-02
8.58041421e-02 -1.85398851e-02 -2.01680344e-02 5.81668504e-02
9.49720368e-02 8.68619047e-03 -1.10520925e-02 -2.13644784e-02
1.17103858e-02 -1.38579868e-02 1.19233401e-02 2.64742766e-02
-2.23848820e-02 1.22723577e-04 -3.45518850e-02 2.53633950e-02
3.13297026e-02 4.28087711e-02 2.94207614e-02 -1.82314962e-02
1.32388491e-02 3.09947096e-02 -9.06949937e-02 1.72531549e-02
-1.94001254e-02 3.73058543e-02 2.29968876e-02 -7.08247647e-02
-1.39049571e-02 -9.69083165e-04 1.17215887e-02 -7.10179433e-02
2.51297053e-07 -1.39412815e-02 7.07565155e-03 -8.92848521e-03
1.10410079e-02 1.87253077e-02 2.12269276e-02 -2.81214342e-02
2.27614511e-02 3.69848683e-02 -5.49834128e-03 -4.84631322e-02
7.79398978e-02 -1.73099134e-02 2.15901807e-02 2.86326986e-02
4.03634459e-03 4.57127541e-02 -1.19320348e-01 -2.15147268e-02
4.14402428e-04 -3.48091638e-03 -3.38214412e-02 -2.95486711e-02
```


WORD EMBEDDINGS

```
1. 79018211e-02 2. 96021570e-02 6. 20511360e-02 -4. 05781248e-05
```

```
# For word embeddings
!pip install gensim
```

```
-2.84881517e-03 -2.97331605e-02 1.33428313e-02 6.07454069e-02
Collecting gensim
  Downloading gensim-4.4.0-cp312-cp312-manylinux_2_28_x86_64_manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Requirement already satisfied: numpy<1.18.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.6.1)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.2.2)
Downloading gensim-4.4.0-cp312-cp312-manylinux_2_28_x86_64_manylinux_2_28_x86_64.whl (27.9 MB)
9.69095062e-03 6.91118985e-02 -1.28524760e-02 -5.07162884e-03
27.9/27.9 MB 34.0 MB/s eta 0:00:00
```

```
Installing collected packages: gensim
Successfully installed gensim-4.4.0
```

```
# Imports the Gensim downloader.
import gensim.downloader as api
```

```
# Downloads a pretrained GloVe embedding model.
# This model has already learned embeddings from a large Wikipedia dataset.
model = api.load("glove-wiki-gigaword-50")
```

```
# model[king]=Gets the embedding vector for the word king.
# Compares the king vector with every other word vector.
# It calculates and finds the most similar other words .
model.most_similar([model['king']], topn=11)
```

```
[=====] 100.0% 66.0/66.0MB downloaded
[('king', 1.0000001192092896),
 ('prince', 0.8236179351806641),
 ('queen', 0.7839043140411377),
 ('ii', 0.7746230363845825),
 ('emperor', 0.7736247777938843),
 ('son', 0.766719400882721),
 ('uncle', 0.7627150416374207),
 ('kingdom', 0.7542161345481873),
 ('throne', 0.7539914846420288),
 ('brother', 0.749241136627197),
 ('ruler', 0.7434253692626953)]
```

RECOMMENDING SYSTEMS (PLAYLISTS)

```
# Pandas is a Python library used for Reading data
import pandas as pd
```

```
# urllib = Python library for working with URLs
# Inside urllib there is a module called request. It knows how to open websites
from urllib import request
```

```
# Go to request module and open and download that link
data = request.urlopen('https://storage.googleapis.com/maps-premium/dataset/yes_complete/train.txt')
```

```
# read the file and now you get raw bytes
# converts bytes into text (utf-8) is the encoding format.
# split into lines (song by song) Every line becomes one list element.
# Line 0 and 1 contains metadata (details like id , name , artist) so ignore it and starts from 2
```

```
lines = data.read().decode("utf-8").split('\n')[2:]
```

```
# Only keep playlists having more than one song.
# Remove playlists with only one song
playlists = [s.rstrip().split() for s in lines if len(s.split())
> 1]
```

```
# Load song metadata
# Download the song information file from the internet and store it in songs_file.
songs_file = request.urlopen('https://storage.googleapis.com/maps-premium/dataset/yes_complete/song_hash.txt')
```

```
# Reads the downloaded file, converts it into readable text, and splits it into separate lines, storing each line as an e
songs_file = songs_file.read().decode("utf-8").split('\n')
```

```
# Removes extra whitespace from each line, s
# plits each line into id, title, and artist using the tab (\t) character,
# stores the result as a list of lists
songs = [s.rstrip().split('\t') for s in songs_file]
```

```
#Creates a Pandas DataFrame from the songs list and names the columns as id, title, and artist.
songs_df = pd.DataFrame(data=songs, columns = ['id', 'title', 'artist'])

# Sets the id column as the index of the DataFrame so each song can be identified by its ID.
songs_df = songs_df.set_index('id')

# Prints the playlist, with each playlist containing a list of song IDs.
```